Where the op1 lean sits: a reanalysis of ex-2.2.11's stored runs

> The op1 lean under the handover recipe is built by the blocks and read by the readout: an untied readout sets the readout vectors of the colors at $-e_1$ and those of the syntax words at $+e_1$, so the e_1 coordinate of a state votes *a syntax word comes next*, and at op1 that vote is more than half of the margin. What the whole-line labeller adds reaches op1 by a small route at most, and the measurements here cannot confirm or rule it out.

This is a reanalysis of ex-2.2.11's 54 stored runs, asking where the containment rise ($\bar{\alpha}$ at op1) comes from. The lean is absent from the embedding table. It grows block by block, and at the last block it sits at the three positions whose next token is a syntax word, and nowhere else. The tied readout cannot move the color embeddings, and gets a smaller lean through the syntax embeddings alone. Whether the readout recruits the axis or adapts to states that already lean, these measurements cannot say. For the whole-line labeller we looked for the pull landing on op1 through lines labelled by their answer. On whole lines it does not, and on lines that a training crop cuts short it does, at a rate too small to show in the per-color comparison.

The readout vector for a token is the one that scores that token as the *next* token: the readout vector for a color is dotted with the state at position i when the model predicts the token at $i + 1$.

Observations

Each line is a measurement on stored runs, with no gate. None is a result; the closing section says what a preregistered follow-up would test.

- **E1** Where in the stream: $\bar{\alpha}$ at op1 is near zero at the embedding on every condition and grows with depth. Every anchored condition leans at op1 by the last block, and their seed ranges overlap.
- **E2** Which positions: at the last block the lean sits at op1, op2, and the answer, the positions whose next token is a syntax word (the op word, `=`, `⏎`). At the op word, `=`, and `⏎` it is near zero on every condition.
- **E3** The readout table: with a readout of its own, the model sets the readout vector for every color at about −0.2 on e_1 and the readout vectors for the syntax words at +0.1 to +0.2, red and non-red colors alike. The tied readout moves only the syntax embeddings. The embeddings of the non-red colors are flat on every condition.
- **E4** What e_1 contributes: removing the e_1 coordinate from the state's contribution to the logits lowers the syntax-against-color log-odds at op1 by more than half on `handover`, by about a third on `handover-slot`, and by a fraction of a nat on `handover-tied` and on the control.
- **E5** Between seeds: within `handover` and within `handover-slot`, the seeds with a wider readout gap lean more at op1. The two conditions have overlapping gaps and different leans, so the gap does not account for what the whole-line labeller adds.
- **E6** Per color: every color leans. A straight-line fit of each color's lean against its redness has a positive intercept (the lean of a color with no red in it) and a positive slope. The rise from the slot labeller to the whole-line one is about the same on every color, and the embedding of a color predicts its lean better than its exposure to answer-earned labels does.
- **E7** The whole-line labeller: on whole lines with two non-red operands and a red answer, the pull puts almost none of its weight on op1. Training crops cut about one line visit in seven short, and on a line cut before its answer the pull can only land on the prompt, so over the crops op1 takes a few percent of such a line's pull. The per-color exposure check is the test of that route, and it is weak.

Scope

This is a reanalysis of stored artifacts, planned in the backlog item: no training, no gates, no verdicts. $\bar{\alpha}$ at op1 is the mean over all 216 grid colors of the cosine between the state of a color at the first operand and e_1, the axis that *red* is anchored to. As ex-2.2.9 defined it, the mean over all five slices, it is 0.02 on the un-anchored control of ex-2.2.11, 0.28 on `handover`, and about half of that when either of the two changes the handover made is undone: 0.18 with the slot labeller back (`handover-slot`), 0.16 with the readout tied again (`handover-tied`). This report reads the same alignment slice by slice, and its figures are at the last block unless they say otherwise, where every anchored condition sits higher. Every experiment since ex-2.1.8 gated the statistic at 0.1, except the handover experiments, which report it without a gate because nobody has said what produces it.

Two mechanisms have been proposed, and each has been checked once before. For the readout: the untied readout frees the embedding table from the logit pressure that kept non-red colors off e_1. Ex-2.2.10 measured the non-red embeddings and found them flat on every condition, untied included. For the labeller: a line labelled through its answer has its pull land on op1. Ex-2.2.12 tried the per-line split on `mix`, where a mean is red only when an operand is, so the answer-labelled group was empty.

The second check cannot be run as a per-line split of the op1 state. The *probe lines* are the fixed lines every run was scored on after training, each fed to the model as its own six-token sequence, so with causal attention the state above op1 is a function of that token alone: two probe lines with the same op1 color have the same op1 state, whatever their answers. (In training the lines are packed, and op1 also sees the lines before it.) What can be measured is where the pull lands on those lines, which the stored alignment maps give, and whether the colors that are exposed to answer-earned labels lean more than those that are not.

Why

Every handover experiment reports the containment statistic without a mechanism for it, and the review asked twice for the why. A drift that costs nothing on the non-red lines is a nuisance; a drift with a mechanism says which change to the recipe would remove it and whether the D2.3 experiments should expect it under their grammars. The stored runs hold the embedding and readout tables, the alignment at every slice and position on every probe line, and the checkpoints, which is enough to say where the lean sits and what the readout does with the axis.

The runs

Every run is one of ex-2.2.11's final checkpoints, with the eval arrays and the run-table entry the experiment published beside it.

condition	readout	span	seeds	what it is
`control`	untied	whole line	5	un-anchored
`handover`	untied	whole line	20	untied readout, whole-line labeller
`handover-slot`	untied	prompt	20	untied readout, slot labeller
`handover-tied`	tied	whole line	9	tied readout, whole-line labeller

The runs. *Every seed of each of ex-2.2.11's four conditions. The span is the set of positions the anchor term pools over on a labelled line: the four prompt roles under the slot labeller, and the whole line, answer and* `⏎` *included, under the whole-line labeller. Under the whole-line labeller a line can also earn its label through its answer.*

The measurements

Three kinds of stored artifact are read.

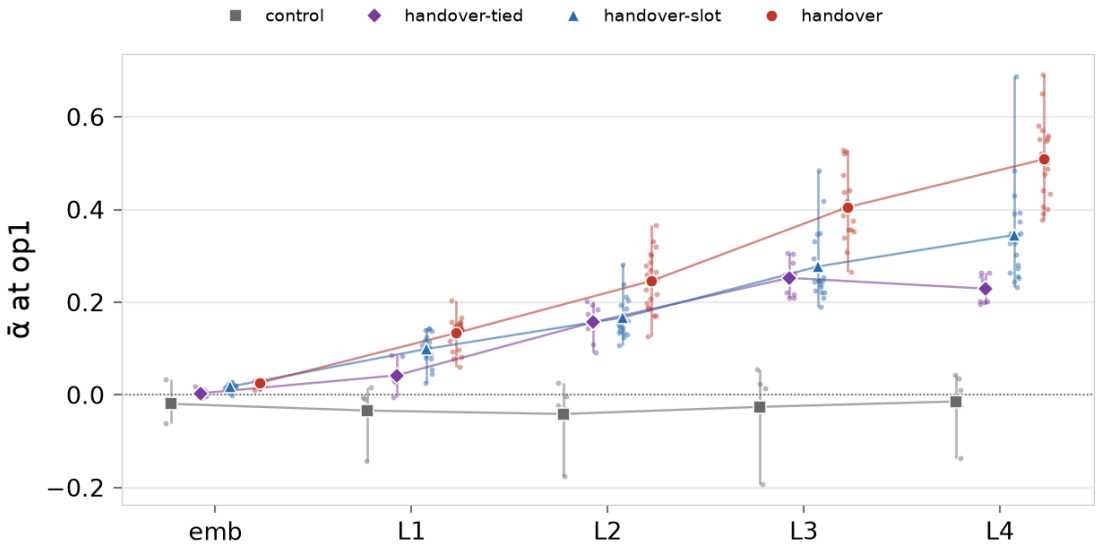
- The **run table** that ex-2.2.11 published holds, per run, the mean alignment with e_1 at every slice and position over the `mix` probe lines, and the e_1 component of the embedding and the readout vector for every token (one and the same vector when the readout is tied).
- The **eval arrays** hold the alignment of each color at op1 at every slice, and the alignment of every position of every probe line of each op, from which the softmin weights that the anchor term would assign are recomputed.
- The **checkpoints** give the logits on the `mix` probe lines. nGPT's logits are $s_z\, hR^\top$, linear in the state h, so the part every logit owes to the e_1 coordinate is $s_z\, h_1 R_{\cdot 1}$ and can be taken out and the log-odds recomputed.

Each figure shows a column of seed dots per condition: one small dot per run, the larger mark the seed mean, and the bar the seed range.[1] The conditions keep ex-2.2.9's inks and markers.

Where in the stream (E1)

What the candidates predict. If the untied readout released the embedding table, the lean would already be there at the embedding and the blocks would carry it along. If the blocks build it, the embedding is flat and the lean grows with depth.

What we saw. The second. At the embedding every condition is within a few hundredths of zero, the control included. The anchored conditions rise from the first block on, and every one of them leans at op1 by the last block. They separate a little with depth, but the seed ranges of neighbouring conditions overlap at every slice.



α̃ at op1 per slice. *Each column is the seeds of one condition at one slice: the mean over the 216 grid colors of the cosine between the state of the color at the first operand and e_1, on the* `mix` *probe lines. The lines join the seed means.*

At the last block the seed means are +0.51 on `handover`, +0.35 on `handover-slot`, +0.23 on `handover-tied`, and -0.01 on the control. The ordering is the one ex-2.2.10 and ex-2.2.11 reported, and it is built by the blocks, since the embedding table starts flat. The gaps between the anchored conditions are smaller than the lean they share: each of them leans at op1 far more than the control does.

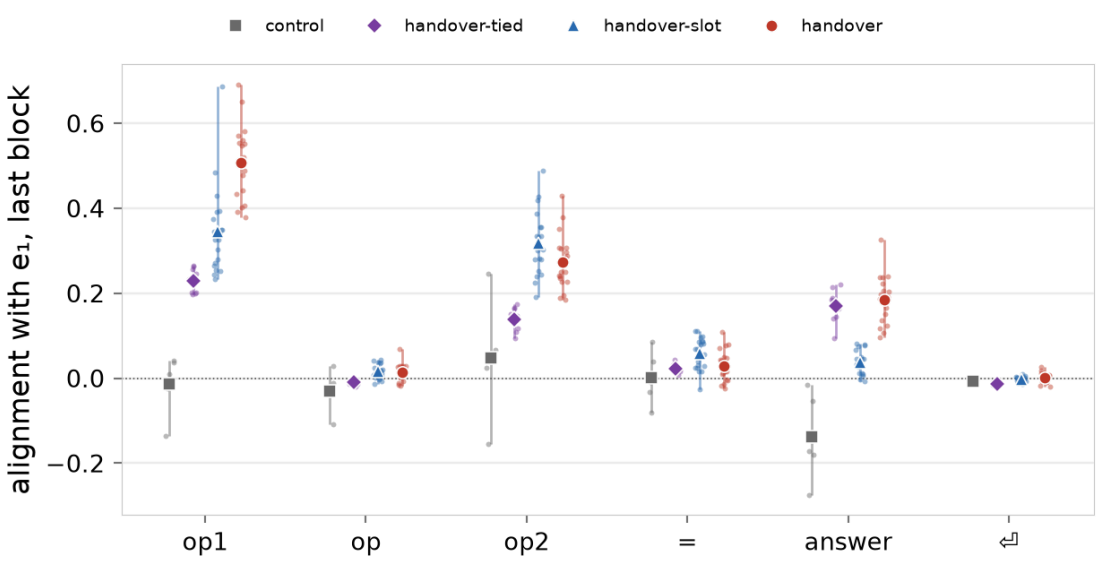
Which positions (E2)

What the candidates predict. A pull that lands on every position of a labelled line would leave a lean at every position, largest where the pull is cheapest. A lean that the readout recruits would sit where the readout has a use for it.

What we saw. At the last block the lean sits at three positions and is near zero at the other three, on every anchored condition. The three are op1, op2, and the answer, whose next token is a syntax word: the op word after op1, `=` after op2, `⏎` after the answer. At the op word, `=`, and `⏎` (whose next token is a color) the lean is within a few hundredths of zero. The pattern is the same on the three anchored conditions and only its size differs. The answer position under the whole-line labeller is the one place the conditions part company, since under the slot labeller the answer is outside the span.

The three leaning positions are also the three color tokens, so this grammar cannot say whether a state leans because its token is a color or because a syntax word comes next: the two go together at every position of every line. The anchor pulls at color positions and the readout reads e_1 as *syntax next*, and both accounts predict this figure. A grammar in which some color is followed by another color would separate them.

The control leans the other way at the answer, and all five of its seeds do, which five coin flips would give one time in sixteen. No term in the control singles out e_1, so we do not build on it.



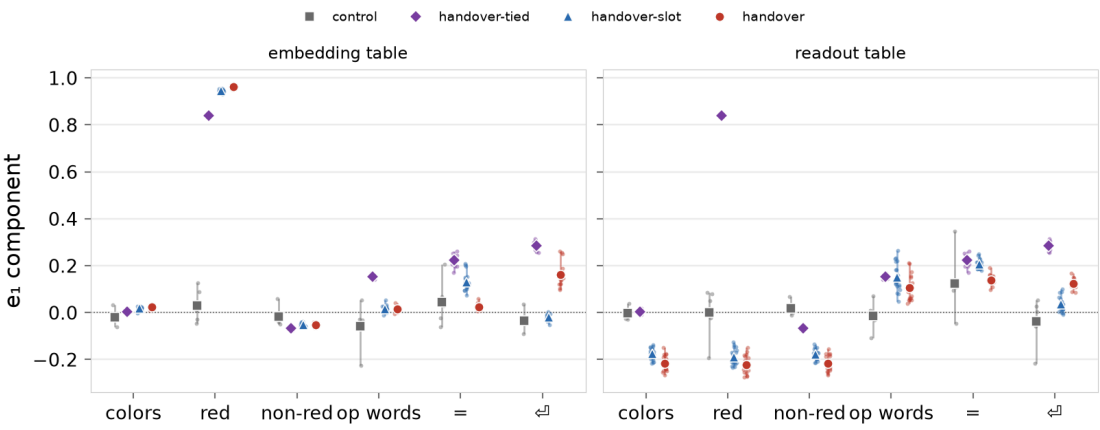
Alignment with e_1 by position, at the last block. *The mean over* `mix`*'s probe lines of the cosine between the state at each position and e_1. The positions alternate between predicting a syntax word (op1, op2, answer) and predicting a color (op,* `=`*,* `⏎`*).*

On `handover` the answer position leans at +0.19 against +0.04 on `handover-slot`; op2 leans at +0.27. The three color-predicting positions average +0.015 on `handover`.

The readout table (E3)

What the candidates predict. Ex-2.2.10 found the non-red embeddings flat on every condition, so if the untied readout is doing something with e_1 it is doing it in the readout table.

What we saw. It is. With a readout of its own, the model sets the readout vector for every color at about −0.2 on e_1, red and non-red colors alike, and the readout vectors for the syntax words at +0.1 to +0.2. The tied condition cannot do the first: in `handover-tied` the readout vectors are the embeddings, and the anchor holds the color embeddings (red at the top, non-red near zero), so only the syntax embeddings move, and they move by about the same amount as under the untied readout. The readout of the control is flat on e_1 for every group, so the split is a response to the anchor rather than a habit of the untied readout. The embeddings of the non-red colors are flat on every condition, as ex-2.2.10 found.



The e_1 component of the tables, by token group. *Left: the embedding table. Right: the readout table, which on* `handover-tied` *is the embedding table again. colors is the mean over all 216 grid colors, red and non-red over the colors at or over the red dose and at or under the non-red dose.*

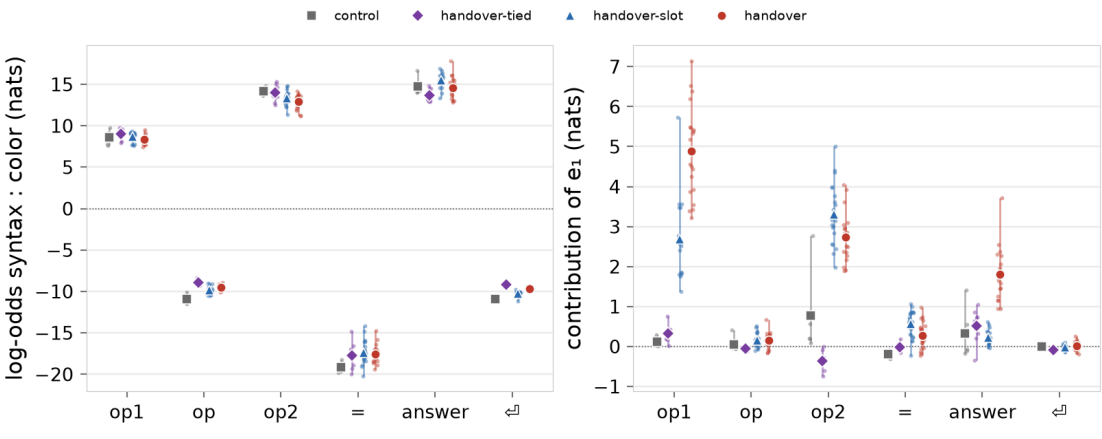
The readout gap (the mean e_1 of the readout vectors for the syntax words, less that of the colors) is +0.34 on `handover`, +0.31 on `handover-slot`, +0.22 on `handover-tied`, and +0.03 on the control. The non-red embeddings sit at -0.05 on `handover` and -0.02 on the control.

What e_1 contributes to the next token (E4)

What the candidates predict. A readout that puts colors at $-e_1$ and syntax at $+e_1$ turns the e_1 coordinate of a state into a vote for *a syntax word comes next*. If the model uses that vote, taking the e_1 coordinate out of the logits should lower the syntax-against-color log-odds at the positions that predict syntax, and the more so the wider the gap.

What we saw. That is what it does. The logits are linear in the state, so the part of every logit that comes from the e_1 coordinate can be removed and the log-odds recomputed from the other 63 coordinates. At op1 the removal lowers the syntax log-odds by 4.9 nats on `handover`, 2.7 on `handover-slot`, and 0.1 on the control. On `handover-tied` the contribution at op1 is a fraction of a nat, about the control's, and at op2 and the answer it goes the other way: the tied table has the red colors at $+e_1$ and the rest of the colors near zero, so the e_1 coordinate of a state there votes for *red* as much as for syntax, and removing it raises the syntax log-odds a little at those positions.

The margin that e_1 contributes to is not the same size everywhere. At op1 the log-odds are +8.3 nats on `handover`, so e_1 supplies more than half of the margin there, and the readout does lean on the axis at that position. At op2 and the answer the log-odds are +13 and +15 nats, of which e_1 supplies +2.7 and +1.8, so there the model is in no doubt about which kind of token comes next whether or not the axis is read. What the measurement says is what the readout has recruited the axis for; that it needs the axis holds at op1 alone.



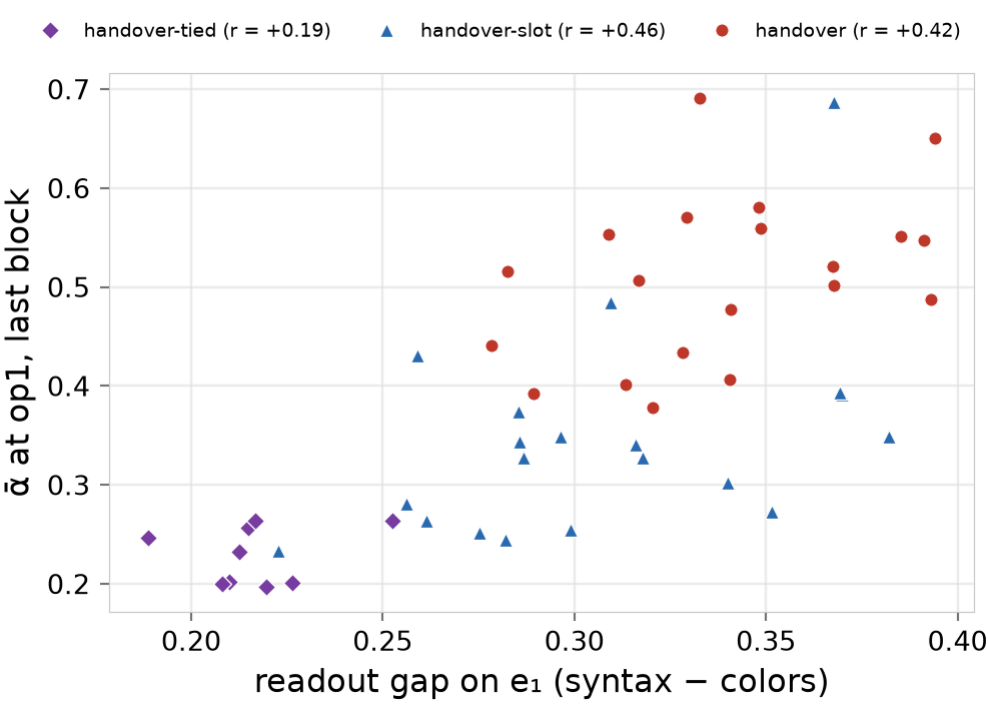
The next-token log-odds and the part e_1 contributes, by position, at the last block. *Left: log P(syntax word next) − log P(color next), mean over the `mix` probe lines. Right: the change in that log-odds when the part of every logit that comes from the e_1 coordinate is removed, so a positive value means e_1 argued for a syntax word.*

At op1, e_1 contributes +4.88 nats on `handover`, +2.68 on `handover-slot`, +0.32 on `handover-tied`, and +0.11 on the control, to a log-odds of about +8 nats.

Between seeds (E5)

What the candidates predict. If the readout gap is what pulls the non-red colors onto the axis, the seeds of one condition that open a wider gap should lean more at op1.

What we saw. They do, within each untied condition. The correlation is moderate: r = +0.42 on `handover` and +0.46 on `handover-slot`, and it is weaker on `handover-tied` (+0.19), whose gap is only in the syntax embeddings. The scatter shows that the two untied conditions occupy overlapping ranges of gap and different ranges of lean: the seeds of `handover` sit a little to the right, and at a given gap they lean more. So the gap tracks what the untied readout adds, and what the whole-line labeller adds is something else.

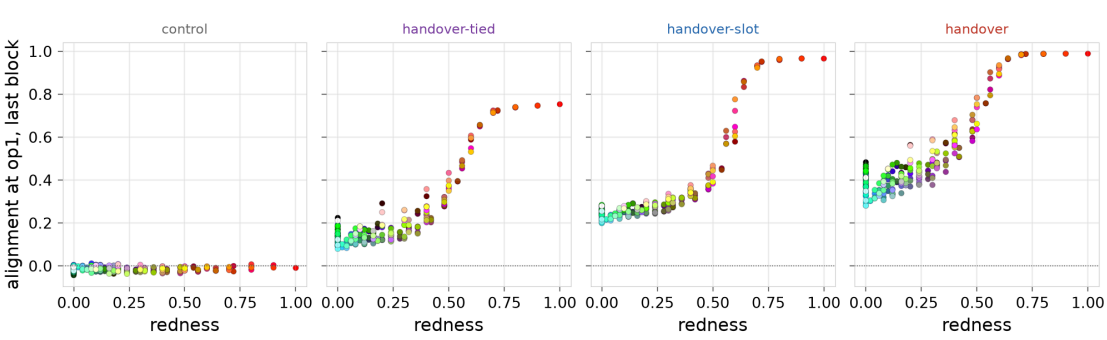


The readout gap against ᾱ at op1, per seed. *Each mark is one run: on the x axis the syntax words' mean e_1 in the readout table less the colors', and on the y axis ᾱ at op1 at the last block. The legend gives the Pearson r within each condition.*

Per color (E6)

What the candidates predict. A lean recruited by the readout as a vote for *syntax next* has no reason to prefer one color over another, so it should be a shift shared by every color, with the grading of the anchor on top of it. A lean that comes from answer-earned labels should be largest on the colors whose lines most often have a red answer.

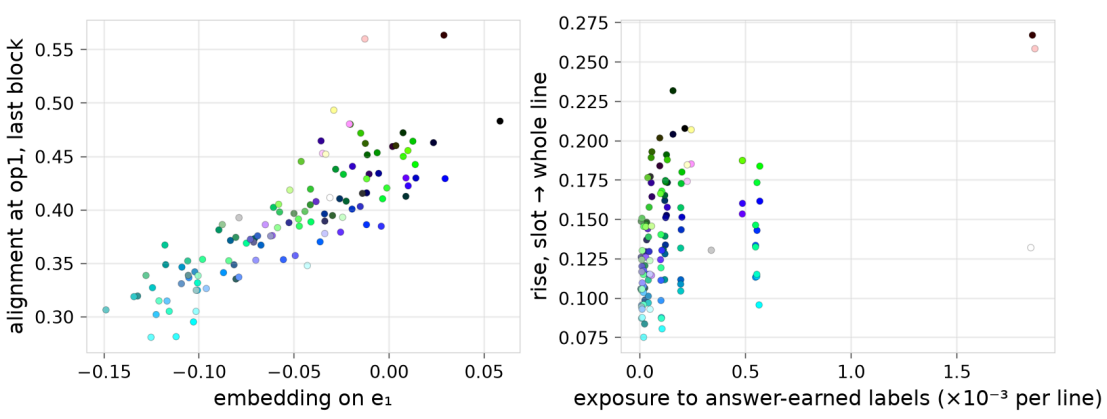
What we saw. Every color leans, and the lean is a shift plus a grade: the non-red colors sit above zero by a nearly constant amount, and the grade with redness is the one the anchor asks for. The rise from `handover-slot` to `handover` is close to the same on every color, so whatever the whole-line labeller adds, it adds to every color alike. The embedding of a color on e_1 predicts its lean well. Its exposure to answer-earned labels predicts the rise less well: the colors form a flat band, and the correlation that remains rests on a few colors at the highest exposure, which are also the ones that rose most.



The alignment of each color at op1 against its redness, at the last block. *One mark per grid color in the color itself, the seed mean per condition. Redness is r(1 − g/2 − b/2) on the unit cube, the grading target.*

A straight line fitted to the last-block lean against redness has an intercept (its value at zero redness) of +0.31 on `handover`, +0.16 on `handover-slot`, and +0.06 on `handover-tied`, with slopes of 0.78, 0.72, and 0.67. The intercept of the control is -0.01.

On `handover` that is a large lean for a color with no red in it, and `handover` is the recipe that ex-2.2.13 replicated and ex-2.2.14 anchored `difference` on. It cost the task nothing measurable (ex-2.2.11 found the paired non-red deficit at 0.004), so the place to expect a cost is intervention: an edit along e_1 at op1 would reach every color by about this much.



What predicts the lean of a non-red color, on `handover`. *Left: the embedding of the color on e_1 against its last-block alignment at op1 ($r = +0.84$). Right: the exposure of the color to labels earned through the answer (its per-line probability as op1, mean over the eleven ops and every partner) against the rise in its lean from `handover-slot` to `handover` ($r = +0.42$; the rise has a spread of 0.04 over colors). Non-red colors only, seed means.*

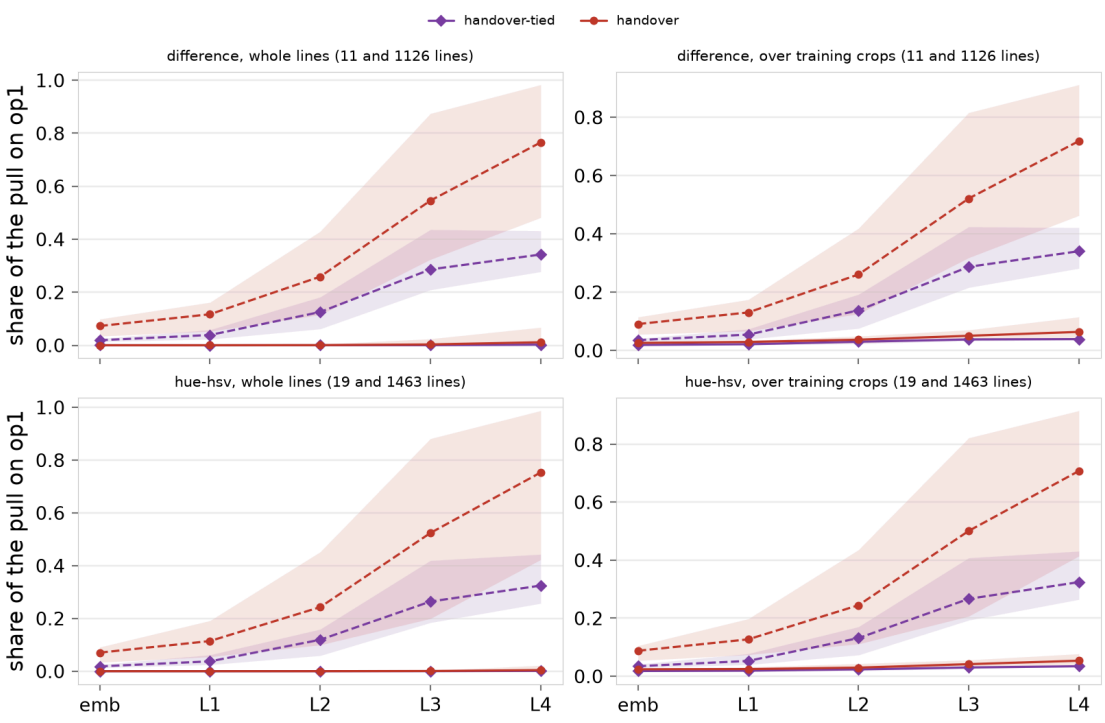
The left panel is the embedding table doing what a table does: a color whose embedding has a little e_1 arrives at the first block with it, and the blocks amplify it. It says where the lean enters the stream, and says nothing of what sets it, since the embedding and the state are trained together. The right panel is the test of the labeller mechanism as a per-color prediction, and it is weak either way: the exposure varies ten-fold across the non-red colors, most of the band is flat, and the correlation that remains rests on a few colors at the highest exposure. Every route by which an answer-earned label reaches op1 scales with this exposure, the cut-short lines of the next section included, so this panel is the test of all of them, and it neither shows the mechanism nor rules it out.

The whole-line labeller (E7)

What the candidates predict. Under the whole-line labeller a line with two non-red operands can earn its label through a red answer, and the pull then pools over the whole line, op1 included. Each labelled line has a pull budget of one, which the softmin weights split over its positions. If the pull lands on op1, op1 should take a fair share of that budget on such lines.

What we saw. On whole lines op1 takes almost none of it, and that is close to built in. The lines are from `difference` and `hue-hsv`, the two ops with the most red answers from two non-red operands (`mix` has none). Both groups have both operands at or under the non-red dose and differ only in the answer. On the answer-labelled lines the answer is aligned with e_1 almost perfectly, and a soft minimum at $\tau = 0.1$ then puts nearly all of the weight there, so the share of op1 follows from how the group was chosen.

Training does not always show a whole line. The training crops are 64-token windows at a random offset into the packed corpus, so 15% of line visits are cut short, and the anchor term pools a labelled line over its visible positions alone. On a line cut before its answer (6% of visits) the pull has to land on the prompt, and a line cut right after op1 puts all of it there. Averaged over the ways a crop shows a line, op1 takes 6.4% of the pull on the answer-labelled `difference` lines at the last block on `handover` (5.4% on `hue-hsv`), against 1.1% on whole lines.



The share of the pull that lands on op1, on lines that can earn a label only through their answer. *Probe lines with both operands non-red, split by a red answer (solid) or a non-red one (dashed), on the two conditions with the whole-line labeller. Top row `difference`, bottom row `hue-hsv`; the line counts are in the panel titles. Left: the softmin weight the anchor term assigns op1 on the whole line. Right: the same weight averaged over the ways a training crop shows the line, where a crop that cuts the line before the answer leaves the pull to the prompt. The shares are read from the final checkpoint; the unlabelled lines receive no pull, so theirs is the share the term would assign if they did. Lines are seed means and bands the seed range.*

So the pull through the answer does reach op1, by one route: lines cut short. The route is small. It is a few percent of the pull on lines that are themselves rare for a non-red op1 (the exposure in the previous figure is of order 10^{-3} per line), and it scales with that exposure, which predicts the rise only weakly. Three caveats. The shares are read from the final checkpoint, and during training they may have been different. The states of a cut line are taken from the probe lines, which assumes that the lines before it in a crop do not change them. And the answer-labelled groups are small, 11 and 19 lines. The alignment at op1 is the same on the two groups by construction (as the Scope section says), so it is not plotted.

What the labeller does visibly change is the answer position, which under the whole-line span the pull *does* land on, and the readout vector for `⏎`, the token that position predicts. On `handover` the answer state leans and the `⏎` readout vector sits further up e_1 than on `handover-slot`; on `handover-slot` the answer is outside the span, its state stays near zero, and the `⏎` readout vector barely moves. That is one more position at which the readout is asked to read e_1 as *syntax next*, and it may be most of what the labeller adds: an axis the readout uses at three syntax-predicting positions rather than two is a stronger feature, and op1, the position with the least context, inherits more of it. This report does not test that; the closing section says what could.

What we make of it

The rise in $\bar{\alpha}$ at op1 looks like the readout recruiting the anchor axis. The anchor puts *red* on e_1 at the operand positions. With a readout of its own, the model finds that a component along e_1 separates the colors from the syntax words in the readout table, and since every color state already has some e_1 on the red lines, it sets the readout vectors for the colors to $-e_1$ and those for the syntax words to $+e_1$. From then on, the e_1 coordinate of a state is read as *a syntax word comes next*, and the positions where that is the prediction (op1, op2, the answer) drift up the axis, every color alike, on top of the grading the anchor asks for. The tied readout cannot set the readout vectors for the colors, because they are the color embeddings and the anchor holds them, so it gets a smaller version of the same thing through the syntax embeddings alone. The whole-line labeller adds the answer position to the set the pull lands on and the readout vector for `⏎` to the set the readout moves, and the lean at op1 grows with it. It also reaches op1 directly on training lines that a crop cuts short, at a few percent of the pull on answer-labelled lines.

Three things this report does not say. It does not say the lean costs the task anything: ex-2.2.11 found the paired non-red deficit at 0.004. It does not say the readout needs e_1 for the syntax prediction everywhere: at op1 on `handover` e_1 supplies more than half of the margin, and at op2 and the answer a small part of a much larger one. And it does not say what the whole-line labeller adds is the answer position; that is the candidate with the most support, and it is untested.

What could be tested. The readout is cheap to test, because it is a table. A pilot on the `handover` recipe that holds the readout vectors for every non-red color and every syntax word off e_1 (the readout-side analogue of `clean_embedding_rows`, applied on the same schedule) should bring $\bar{\alpha}$ at op1 down toward the level of `handover-tied` without touching *red*, and it should take the e_1 contribution to the syntax log-odds with it. The labeller wants the span narrowed by one position: a `handover` arm whose whole-line labeller keeps the keying but pools over every role except the answer (op1, the op word, op2, `=`, and `⏎`), or over the prompt roles alone, would say whether the answer position is what the whole-line span adds. That is a different arm from the one the note on ex-2.2.14 proposed, which drops op1 from the span instead; the two answer different questions. A third arm would close the route through cut lines: skip the pull on a line the crop cuts before its answer, or pool with a bias toward later positions. Each is one condition on the existing setup, and each reads $\bar{\alpha}$ at op1 beside the two arms this report has already characterised.

An attention mask that resets at `⏎` would isolate the lines in training as the probe lines are isolated at eval. It would also free `⏎` from predicting the first color of an unrelated line, which may change how the readout uses e_1 at the answer. The D2.2 pivot proposes that mask for its own reasons.

Method

Sources. Ex-2.2.11's four conditions at every stored seed: the checkpoint, the eval arrays, and the run-table entry per run, and the probe lines every run was scored on. The report reads this pass's outputs through `project_store()`.

Alignment by position. From the run table: the mean over the `mix` probe lines of the cosine between the state at each position and e_1, at each of the five slices, as ex-2.2.9's eval pass computes it. At op1 the mean over lines is the mean over the 216 colors, since each color is op1 on the same number of lines.

Tables. The e_1 component of the embedding and the readout vector for every token, from the run table, summarized by token group: the 216 colors, those at or over the red dose (0.8) and at or under the non-red dose (0.2), the eleven op words, `=`, and `⏎`. The readout gap is the mean over the three syntax groups less the mean over the colors.

Contribution of e_1. On the `mix` probe lines, from the checkpoint: the logits $s_z\, hR^\top$ at every position, and the log-odds $\log \sum_{\text{syntax}} p - \log \sum_{\text{colors}} p$. The contribution is that log-odds less the log-odds of $s_z\, hR^\top - s_z\, h_1 R_{\cdot 1}$, the logits with the part from the e_1 coordinate removed. Nothing is renormalized between the two, so the difference is what that coordinate contributed.

Per color. From the eval arrays: the alignment of each grid color at op1 at each slice, on `mix`. The fit against redness is ordinary least squares over the 216 colors of the seed mean.

Exposure. From the grammar and the model of the labeller (each slot draws a label at redness^8 × 0.04): for each color as op1, the mean over the eleven ops and every partner of the probability that the line is labelled through op1, through op2 when op1 did not draw, and through the answer alone, with the answer's redness taken in expectation over the stochastic rounding.

The labeller's lines. Probe lines of `difference` and `hue-hsv` from walk 0, the part of the probe set that walks every grid color through op1 (the order-sensitive ops also have a walk 1 with the color as op2). The lines kept have both operands at or under the non-red dose, and are split by an answer at or over the red dose or at or under the non-red dose. The share on whole lines is `softmin_weights(1 - α, τ = 0.1)` over the condition's span (the four prompt roles or the whole line), read at op1.

Training crops. The share over training crops follows the batch sampler of ex-2.2.11: 64-token windows onto the packed corpus at a uniform offset, a tenth of them with a zeroed prefix of 1 to 20 tokens. Enumerating every offset and prefix gives how often a line is seen whole or as each run of roles (85.3% whole, 1.5% for each of the ten cut runs). For each run that includes a pulled position, op1 takes the softmin weight over the visible pulled roles if the run starts at op1, and nothing otherwise; the share is the average over runs, weighted by how often each occurs. A run that starts at op1 is a prefix of the line, so under causal attention its states are the probe line's own, lines earlier in the crop aside.

Where the data is. The run table is under the `metrics` ref of `reports/m2/op1-lean` and every array under `arrays`, keyed by run label and name, with the exposure model's arrays under `grammar/`.

---

1. `control` has five seeds, `handover-tied` nine, and the other two twenty, so a control column's range is a rougher measurement of its spread than the others'. ↩